Guide SSU Bokengi — Trajets sécurisés et secteurs de risque

Documentation du calcul des itinéraires, des pourcentages par secteur et de la carte de prédiction géospatiale

Généré le 25/06/2026 à 23:58 — 1.0 — prediction-service + prediction_shared

Chaîne de traitement

• Export signalements V → data_reports.csv (Celery / manage.py)

• Entraînement ML → best_model.pkl + scaler.pkl (prediction_shared)

• prediction-service charge CSV + modèle (volume Docker partagé)

• Frontend carte → GET /api/danger-predictions/ → proxy Django → GET /grid/layers

• Trajet sécurisé → Mapbox Directions → POST /api/safe-routes/score/ → POST /routes/score

1. Vue d'ensemble

L'application SSU Bokengi propose deux visualisations complémentaires : une **carte de chaleur** (grille 48×48) alimentée par le microservice *prediction-service*, et des **secteurs** (8×8) affichant des pourcentages faible / modéré / élevé. Le mode **trajet sécurisé** interroge Mapbox pour obtenir plusieurs candidats (marche / vélo), puis les classe selon un score de sécurité combinant modèle ML et signalements validés terrain (statut V).

2. Données sources

Tous les calculs s'appuient sur **data_reports.csv** : export des signalements validés (latitude, longitude, degré F/M/E, description, date). Le modèle ML (Gradient Boosting ou meilleur candidat en CV) est entraîné sur ce fichier. À l'inférence, six features sont utilisées : latitude, longitude, heure, jour de la semaine, densité locale (500 m) et degré moyen des voisins (BallTree haversine).

3. Grille 48×48 et score de risque par cellule

Kinshasa est découpée en une grille régulière de 48×48 points (~2 304 cellules). Pour chaque cellule, le service calcule la densité de signalements dans un rayon de **500 m** et le degré moyen observé chez les voisins. Le modèle produit trois probabilités (faible, modéré, élevé). Le **risk_score** composite (0–1) combine probabilités modéré/élevé et contexte local :
risk_score = min(1, proba_modéré×0,45 + proba_élevé×0,95 + facteur_densité×0,15 + facteur_danger×0,15)
Seules les cellules avec signal significatif sont affichées comme points cliquables (filtre anti-saturation verte).

4. Secteurs 8×8 — découpage et pourcentages

La ville est aussi découpée en **64 secteurs** (8 lignes × 8 colonnes) sur la bbox OSM de Kinshasa. Pour chaque secteur, deux jeux de statistiques sont calculés :

A) Statistiques terrain (prioritaires) — si au moins un signalement validé se trouve dans le secteur, les pourcentages affichés (pct_faible, pct_modere, pct_eleve) reflètent la répartition **réelle** des degrés F/M/E des signalements.

B) Statistiques grille ML (secours) — s'il n'y a aucun signalement validé dans le secteur, les pourcentages proviennent de la répartition des prédictions ML sur les cellules 48×48 du secteur.

Le **niveau dominant** (risk_level : faible / modere / eleve) est dérivé des pourcentages retenus et du score moyen, avec seuils : élevé si pct_élevé ≥ 25 % ou score ≥ 0,65 ; modéré si pct_modéré ≥ 30 %, pct_élevé ≥ 10 % ou score ≥ 0,40.

5. Trajet sécurisé — pipeline

Étape 1 — Candidats Mapbox : le frontend appelle l'API Directions (profils walking et cycling, alternatives=true) entre origine et destination. Jusqu'à 10 routes candidates sont conservées (dédoublonnage géométrique).

Étape 2 — Échantillonnage du tracé : chaque route est découpée en jusqu'à **48 points** régulièrement espacés le long de la polyligne, plus l'arrivée exacte. Les signalements validés situés à ≤ **120 m** du tracé sont ajoutés comme points de contrôle supplémentaires (pour ne pas « rater » un passage sur un signalement).

Étape 3 — Score ML par point : pour chaque point échantillon, prediction-service calcule densité,

danger voisin et prédiction ML (predict_point).

Étape 4 — Hazard terrain : indépendamment du ML, un score **report_hazard** (0–1) mesure la proximité des signalements validés réels (voir tableau distances).

Étape 5 — Agrégation route : moyenne et maximum des risques ML et terrain, comptage des segments à haut risque, puis calcul du **optimization_cost** (plus bas = plus sûr). La route recommandée est celle au coût minimal.

6. Formule de classement des itinéraires

Pour chaque route candidate :
combined_avg = max(moyenne_risk_score_ML, moyenne_report_hazard × 0,85)
combined_max = max(max_risk_score_ML, max_report_hazard)
report_penalty = max(max_hazard×0,55, avg_hazard×0,35) — renforcée à ≥ 0,65 si passage à ≤ 120 m d'un signalement.
optimization_cost = min(1, combined_avg×0,42 + combined_max×0,28 + high_risk_penalty + report_penalty + distance_penalty + duration_penalty)
safety_score = 1 − optimization_cost (affiché en % côté frontend).

Niveau textuel : **low** / **medium** / **high** selon le score combiné et la proportion de points à haut risque.

7. Carte de chaleur — couleurs

Trois couches heatmap Mapbox superposées, filtrées par classe prédite :
• **Vert / bleu** — prédiction faible (danger=0)
• **Orange / ambre** — prédiction modérée (danger=1)
• **Rouge** — prédiction élevée (danger=2)

Les poids heatmap sont calibrés pour que modéré et élevé ne produisent pas la même teinte. Les cercles cliquables reprennent le code couleur vert / jaune / rouge selon la classe.

8. API et déploiement

• **GET /grid/layers** — grille, heatmap, secteurs (prediction-service)
• **POST /routes/score** — scoring des candidats Mapbox
• **POST /predict** — prédiction ponctuelle
• Django (web) délègue l'inférence au microservice ; seule l'agrégation par commune reste côté backend (requête PostgreSQL).
Volumes Docker partagés : *ssu_backend/prediction/data* et *ssu_backend/prediction/model*.

Annexe A — Formules clés

Features ML (6) : `lat, lon, hour, day_of_week, density_500m, mean_neighbor_degree`

risk_score cellule : `min(1, P(modéré)×0,45 + P(élevé)×0,95 + min(density/8,1)×0,15 + min(mean_danger/2,1)×0,15)`

safety_score trajet : `1 − optimization_cost`

Distances hazard terrain (report_proximity_hazard)

Distance au signalement	Effet sur hazard_score
≤ 50 m (sur le point)	0,92 – 1,00 (selon degré F/M/E)
≤ 120 m	0,72 – 0,90
≤ 350 m	0,45 – 0,70
≤ 500 m (zone BallTree)	Décroissance linéaire jusqu'à ~0,25–0,60
> 500 m	0 (hors voisinage)

Pénalités dans optimization_cost (trajet)

Composante	Formule / seuil
Risque moyen combiné	combined_avg × 0,42
Pic de risque	combined_max × 0,28
Segments haut risque	(high_risk_points / N) × 0,22
Signalements proches	report_penalty (voir §6)
Distance	min(distance/15000, 1) × 0,12
Durée	min(duration/3600, 1) × 0,05

Secteur — champs renvoyés à l'API

Champ	Description
pct_faible / modere / eleve	Pourcentages affichés (terrain ou ML)
grid_pct_*	Pourcentages ML bruts (toujours calculés)
observed_report_count	Nombre de signalements validés dans le secteur
avg_risk_score	Score moyen des cellules grille dans le secteur
avg_density	Densité moyenne signalements / 500 m
risk_level	faible | modere | eleve

Annexe B — Limites et bonnes pratiques

• Mapbox ne reçoit pas de zones d'évitement : le contournement repose uniquement sur le classement des alternatives déjà proposées par Directions.

• Un trajet unique imposé par la topographie peut rester risqué si toutes les alternatives passent près de signalements.

• Les pourcentages secteur « terrain » ne reflètent que les signalements validés (V), pas les signalements en attente.

• La grille ML peut sous-estimer un point isolé si le modèle généralise « faible » — d'où la priorité aux stats terrain dans les secteurs peuplés de signalements.

• Performance : grille 48×48 ≈ 2 300 inférences ; timeout API recommandé 90 s.